

確保工作場所安全偵測系統安全無虞

來源：<https://codelabs.developers.google.com/next26/brk3-034-securing-workplace-safety?hl=zh-tw>

步驟數：10

目錄

1. [簡介](#)
2. [事前準備](#)
3. [建立 Assured Workloads 基礎](#)
4. [設定 GKE 基礎架構](#)
5. [建立及設定 GKE 叢集](#)
6. [應用程式身分設定](#)
7. [建構及部署應用程式](#)
8. [測試應用程式](#)
9. [清除](#)
10. [恭喜](#)

1. 簡介

在本程式碼實驗室中，您將建構基本示範應用程式，說明如何整合各種 Google Cloud 服務，以用於受法規遵循限制的專案。這個專案使用下列安全性功能：

- [資料邊界](#)，協助解決受監管環境的控管問題
- 客戶自行管理的加密金鑰 ([CMEK](#))，用於保護靜態資料；
- [GKE 受防護的節點](#)和 [GKE 機密節點](#)，有助於在處理期間保護資料安全；
- [GKE 適用的 Workload Identity](#)，可管理 Vertex AI 的存取權，並利用 Gemini 執行圖片處理工作。
- [Vertex AI 生成式 AI](#)，可管理影片串流中的安全帽偵測作業

本程式碼研究室適合各種程度的開發人員，包括初學者。您將使用 Google Cloud Shell 中的指令列介面和 Python 程式碼。您不需要是 Python 專家，但瞭解如何解讀程式碼有助於理解概念。

注意：這是簡化的概念驗證，並非正式版應用程式。在實際情況中，對這項服務採取額外防護措施，例如通過驗證的外部存取權，以及安全的外部存取權。

步驟 2 · 約 5 分鐘

2. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

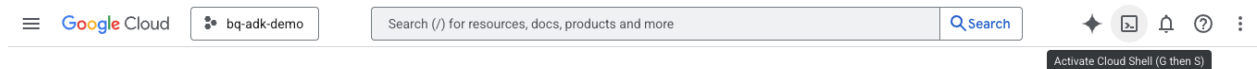
專案設定

../shared/_project-setup.md

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 在 Google Cloud 控制台頂端點選「啟用 Cloud Shell」：



2. 連線至 Cloud Shell 後，請執行下列指令，在 Cloud Shell 中驗證您的驗證：

〇〇〇〇

```
gcloud auth list
```

3. 執行下列指令，確認專案已設定為搭配 gcloud 使用：

〇〇〇〇

```
gcloud config get project
```

4. 確認專案符合預期，然後執行下列指令設定專案 ID：

〇〇〇〇

```
export PROJECT_ID=$(gcloud config get project)
```

必要 IAM 權限

您用於本程式碼研究室的帳戶必須具備下列 IAM 角色。您必須具備這些權限，才能建立必要的 Google Cloud 資源 (專案、資料夾、GKE 叢集、KMS 金鑰、服務帳戶等)，並設定 Assured Workload。

機構層級：

- roles/assuredworkloads.admin (**Assured Workloads 系統管理員**)：建立及管理 Assured Workloads 資源本身，確保符合法規設定。

在帳單帳戶層級 (位於帳單帳戶中)：

- roles/billing.accountUser (帳單帳戶使用者)：連結指定的帳單帳戶。

設定 Codelab 變數

如要建立所需基礎架構，請提供下列環境變數：

```
# The ID of the Billing Account in the format (XXXXXX-XXXXXX-XXXXXX).
# This value will be used to attach in the projects created using Assured Workloads
export BILLING_ACCOUNT=

# The ID of a Google Cloud Platform organization
# Run `gcloud organizations list` to check all your available organizations
export GCP_ORGANIZATION=

# The numeric ID of a folder where the Assured Workloads will create the resources.
export FOLDER_ID=

# Region where the application will be deployed.
# Since you are using Assured Workloads, you MUST use one of the valid locations as described
# here: <https://docs.cloud.google.com/assured-workloads/docs/locations>
export REGION="us-central1"

# The ID of an existing Google Cloud project to be used for API quota and billing purposes.
# This project will only be used to enable the Assured Workloads API and create an Assured
# Workload.
export QUOTA_PROJECT_ID=

# Random suffix used to avoid naming collisions when creating the GCP projects.
export RANDOM_SUFFIX=$(cat /dev/urandom | tr -dc 'a-z0-9' | head -c 5 ; echo)

# The ID of the projects that will be created using Assured Workloads.
# You can modify this value if you want a custom id.
export PROJECT_ID="il5-gemini-vision-aw-${RANDOM_SUFFIX}"
export KMS_PROJECT_ID="il5-gemini-vision-kms-${RANDOM_SUFFIX}"
```

步驟 3 · 約 5 分鐘

3. 建立 Assured Workloads 基礎

您現在可以在受監管的环境中，為應用程式奠定基礎。使用 **Assured Workloads** 為資源建立受控環境，協助強制執行法規遵循要求。

設定配額專案

在配額專案中啟用 Assured Workloads API。建立及管理 Assured Workloads 時，必須使用這個 API。

```
gcloud services enable assuredworkloads.googleapis.com \
  --project="${QUOTA_PROJECT_ID}"
```

建立 Assured Workloads 環境

下列指令會為示範建立安全的「登陸區」。系統會在您指定的資料夾和帳單帳戶下，建立兩個新的 Google Cloud 專案。

- 一個專案會代管 GKE 叢集和應用程式。
- 另一個專案則管理客戶自行管理的加密金鑰 (CMEK)。

系統會在專案建立時，自動將指定的 IL5 法規遵循控制項套用至兩個專案。

執行下列指令，建立工作負載環境。

```

export ASSURED_WORKLOAD_ID=$(gcloud assured workloads create \
  --project="${QUOTA_PROJECT_ID}" \
  --display-name="DoD IL5 Gemini Vision Demo" \
  --compliance-regime="IL5" \
  --billing-account="billingAccounts/${BILLING_ACCOUNT}" \
  --location="${REGION}" \
  --organization="${GCP_ORGANIZATION}" \
  --provisioned-resources-parent="folders/${FOLDER_ID}" \
  --resource-settings="consumer-project-id=${PROJECT_ID},consumer-project-name=DoD IL5
Workloads,encryption-keys-project-id=${KMS_PROJECT_ID},encryption-keys-project-name=DoD IL5
KMS" \
  --labels="codelab=gemini-vision-demo" \
  --format="value(name)")

echo "Assured Workload created: ${ASSURED_WORKLOAD_ID}"

export WORKLOAD_FOLDER_ID=$(gcloud assured workloads describe ${ASSURED_WORKLOAD_ID} \
  --location="${REGION}" \
  --project="${QUOTA_PROJECT_ID}" \
  --format="json" | grep -B 1 "CONSUMER_FOLDER" | grep -oE "[0-9]{10,}")

echo "Assured Workload folder created: ${WORKLOAD_FOLDER_ID}"

gcloud projects create "${PROJECT_ID}" \
  --folder="${WORKLOAD_FOLDER_ID}" \
  --name="DoD IL5 Workloads"

gcloud billing projects link "${PROJECT_ID}" \
  --billing-account="${BILLING_ACCOUNT}"

export PROJECT_NUMBER=$(gcloud projects describe "${PROJECT_ID}" --
format="value(projectNumber)")

```

啟用必要的 Google Cloud API

建構前，請先啟用所需服務的必要 API。

下列指令會啟用主要工作負載專案中，這個程式碼研究室所需的所有服務。

```

gcloud services enable \
  aiplatform.googleapis.com \
  artifactregistry.googleapis.com \
  cloudkms.googleapis.com \
  compute.googleapis.com \
  container.googleapis.com \
  iam.googleapis.com \
  logging.googleapis.com \
  monitoring.googleapis.com \
  --project="${PROJECT_ID}"

```

步驟 4 · 約 5 分鐘

4. 設定 GKE 基礎架構

建構支援 GKE 叢集的核心基礎架構。包括為叢集設定專屬網路，以及設定自己的加密金鑰，保護節點上的資料。

設定虛擬私有雲網路

建立自訂虛擬私有雲 (VPC) 和子網路。這種方法可讓您完全掌控 IP 位址範圍，並確保叢集如預期般隔離。

執行下列指令，在指定區域中建立虛擬私有雲網路和子網路。

```
export GKE_NETWORK_NAME="il5-gke-network"
export GKE_SUBNETWORK_NAME="il5-gke-subnet"

gcloud compute networks create "${GKE_NETWORK_NAME}" \
  --description="VPC network for GKE cluster in DoD IL5 Assured Workload" \
  --subnet-mode="custom" \
  --project="${PROJECT_ID}"

gcloud compute networks subnets create "${GKE_SUBNETWORK_NAME}" \
  --network="${GKE_NETWORK_NAME}" \
  --range="10.10.0.0/20" \
  --region="${REGION}" \
  --description="Subnet for GKE cluster nodes in DoD IL5 Assured Workload" \
  --project="${PROJECT_ID}"
```

使用 Cloud KMS 設定加密

如要符合嚴格的靜態資料法規遵循規定，請使用客戶自行管理的加密金鑰 (CMEK)。您可以直接控管用來加密 GKE 節點開機磁碟的金鑰。

```
export KMS_KEYRING_NAME="il5_gke_key_ring"
export KMS_KEY_NAME="il5_gke_key"

gcloud kms keyrings create "${KMS_KEYRING_NAME}" \
  --location="${REGION}" \
  --project="${KMS_PROJECT_ID}"

gcloud kms keys create "${KMS_KEY_NAME}" \
  --keyring="${KMS_KEYRING_NAME}" \
  --location="${REGION}" \
  --purpose="encryption" \
  --project="${KMS_PROJECT_ID}"
```

由於加密金鑰位於與需要使用金鑰的服務不同的 KMS 專案中，因此工作負載專案中的 Google Compute Engine 服務代理需要存取金鑰。

您必須明確授予工作負載專案的 GCE 服務代理程式使用這個金鑰的權限。下列指令會將 IAM 政策繫結新增至金鑰，並將必要角色授予服務代理。

```
gcloud kms keys add-iam-policy-binding "${KMS_KEY_NAME}" \
  --location="${REGION}" \
  --keyring="${KMS_KEYRING_NAME}" \
  --member="serviceAccount:service-${PROJECT_NUMBER}@compute-
system.iam.gserviceaccount.com" \
  --role="roles/cloudkms.cryptoKeyEncrypterDecrypter" \
  --project="${KMS_PROJECT_ID}"
```

設定 GKE 節點服務帳戶

```
export GKE_NODE_SA=gke-node-sa

gcloud iam service-accounts create "${GKE_NODE_SA}" \
  --display-name="GKE Node Service Account" \
  --project="${PROJECT_ID}"

gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${GKE_NODE_SA}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/logging.logWriter"

gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${GKE_NODE_SA}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/monitoring.metricWriter"
```

5. 建立及設定 GKE 叢集

現在可以建立 GKE 叢集。下列指令會佈建 GKE 叢集，並啟用多項安全性功能。

Google Cloud 佈建節點和控制層時，這項作業需要幾分鐘才能完成。

```
export GKE_CLUSTER=ppe-app

gcloud beta container clusters create "${GKE_CLUSTER}" \
  --project="${PROJECT_ID}" \
  --region="${REGION}" \
  --service-account="${GKE_NODE_SA}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --release-channel="regular" \
  --machine-type="n2d-standard-4" \
  --image-type="COS_CONTAINERD" \
  --disk-type="pd-ssd" \
  --disk-size="50" \
  --boot-disk-kms-
key="projects/${KMS_PROJECT_ID}/locations/${REGION}/keyRings/${KMS_KEYRING_NAME}/cryptoKeys/${
KMS_KEY_NAME}" \
  --metadata disable-legacy-endpoints=true \
  --num-nodes="1" \
  --network="projects/${PROJECT_ID}/global/networks/${GKE_NETWORK_NAME}" \
  --
subnetwork="projects/${PROJECT_ID}/regions/${REGION}/subnetworks/${GKE_SUBNETWORK_NAME}" \
  --security-posture="standard" \
  --workload-vulnerability-scanning="disabled" \
  --workload-pool="${PROJECT_ID}.svc.id.goog" \
  --workload-metadata=GKE_METADATA \
  --
addons="HorizontalPodAutoscaling,HttpLoadBalancing,NodeLocalDNS,GcePersistentDiskCsiDriver" \
  --max-surge-upgrade=1 \
  --max-unavailable-upgrade=0 \
  --binauthz-evaluation-mode="DISABLED" \
  --no-enable-basic-auth \
  --enable-autoupgrade \
  --enable-autorepair \
  --enable-confidential-nodes \
  --confidential-node-type=sev \
  --enable-ip-access \
  --enable-ip-alias \
  --enable-managed-prometheus \
  --enable-dns-access \
  --enable-shielded-nodes \
  --shielded-integrity-monitoring \
  --shielded-secure-boot
```

連線至新叢集

如要與新叢集互動，請設定本機 kubectl 指令列工具。

這項指令會擷取叢集的憑證和端點，並自動設定本機 kubeconfig 檔案。執行這項指令後，您執行的任何 kubectl 指令都會導向新的 GKE 叢集。

```
gcloud container clusters get-credentials "${GKE_CLUSTER}" \
  --region="${REGION}" \
  --project="${PROJECT_ID}" \
  --dns-endpoint
```

6. 應用程式身分設定

如要將 Kubernetes 服務帳戶連結至 Google Cloud IAM 服務帳戶，請設定 Workload Identity。

建立服務帳戶

在叢集內建立專屬的 Kubernetes 服務帳戶 (KSA)。

```
export GKE_NAMESPACE=default
export GKE_SA=ppe-sa

kubectl create sa "${GKE_SA}" --namespace="${GKE_NAMESPACE}"
```

接著，應用程式需要 Google Cloud 中的身分。為應用程式建立 Google Cloud IAM 服務帳戶。建立服務帳戶後，請授予必要角色。

```
# Create GCP service account
gcloud iam service-accounts create "${GKE_SA}" \
  --project="${PROJECT_ID}"
# Grant necessary roles
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${GKE_SA}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

允許 Kubernetes 服務帳戶模擬 IAM 服務帳戶

建立這兩個服務帳戶後，最後一個步驟是建立兩者之間的連結。這項程序分為兩個部分。首先，請將 IAM 政策新增至 Google Cloud 服務帳戶。

```
# Allow the Kubernetes service account to act as GCP service account by using Workload Identity
gcloud iam service-accounts add-iam-policy-binding
"${GKE_SA}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --project="${PROJECT_ID}" \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:${PROJECT_ID}.svc.id.goog[${GKE_NAMESPACE}/${GKE_SA}]"
```

其次，為 Kubernetes 服務帳戶加上註解。

```
kubectl annotate --namespace="${GKE_NAMESPACE}" serviceaccount "${GKE_SA}" \
  iam.gke.io/gcp-service-account="${GKE_SA}@${PROJECT_ID}.iam.gserviceaccount.com"
```

完成這項設定後，叢集中任何使用 Kubernetes 服務帳戶執行的 Pod，現在都能存取 Vertex API。

注意：您可以消除服務帳戶模擬功能，簡化這項設定。如需詳細資料和限制，請參閱[這裡](#)。

步驟 7 · 約 10 分鐘

7. 建構及部署應用程式

現在要封裝應用程式、儲存，然後部署至 GKE 叢集。

建立 Artifact Registry 存放區

您必須先將應用程式封裝為 Docker 容器，並儲存在 Artifact Registry 存放區中，才能執行應用程式。請使用下列指令建立這個存放區。

```
export REPOSITORY_ID=ppe-repo

gcloud artifacts repositories create "${REPOSITORY_ID}" \
  --repository-format=docker \
  --location="${REGION}" \
  --project="${PROJECT_ID}" \
  --description="Regional Docker repo for PPE App"
```

您必須明確授予 GKE 節點使用的服務帳戶，從新存放區讀取資料的權限。

```
gcloud artifacts repositories add-iam-policy-binding "${REPOSITORY_ID}" \
  --location="${REGION}" \
  --role="roles/artifactregistry.reader" \
  --project="${PROJECT_ID}" \
  --member="serviceAccount:${GKE_NODE_SA}@${PROJECT_ID}.iam.gserviceaccount.com"
```

建構及推送 Docker 映像檔

建構 Docker 映像檔並推送至存放區。首先，請複製原始碼並建構容器映像檔。然後使用完整的 Artifact Registry 路徑標記，並推送至先前建立的存放區。

```
git clone https://github.com/GoogleCloudPlatform/next-26-sessions.git

cd BRK3-034-workplace-safety

cd ppe

export IMAGE_TAG="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPOSITORY_ID}/ppe-app:v15"
docker build -t "${IMAGE_TAG}" .
docker push "${IMAGE_TAG}"
```

部署至 GKE

容器映像檔已存放在 Artifact Registry 中，最後一個步驟是指示 GKE 提取並運作執行該映像檔。方法是在 Kubernetes 資訊清單檔案中定義應用程式的資源，然後套用至叢集。這項指令會建立檔案中定義的 Deployment 和 Service 物件。

這項指令會部署分析端點，供 GKE 叢集外部存取。請確保這個端點無法從不受信任的網路 (例如預設虛擬私有雲) 存取。在正式版應用程式中，建議採用 [Model Armor](#) 等額外安全措施。

```
export GKE_DEPLOY=ppe-detector

cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ${GKE_DEPLOY}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ${GKE_DEPLOY}
  template:
    metadata:
      labels:
        app: ${GKE_DEPLOY}
    spec:
      serviceAccountName: ${GKE_SA}
      containers:
      - name: ${GKE_DEPLOY}
        image: ${IMAGE_TAG}
        env:
        - name: PROJECT_ID
          value: ${PROJECT_ID}
        - name: LOCATION
          value: ${REGION}
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: "250m"
            memory: "512Mi"
          limits:
            cpu: "500m"
            memory: "1Gi"
---
apiVersion: v1
kind: Service
metadata:
  name: ${GKE_DEPLOY}
spec:
  type: LoadBalancer
  selector:
    app: ${GKE_DEPLOY}
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
EOF
```

步驟 8 · 約 5 分鐘

8. 測試應用程式

最後一步是存取及測試試用版應用程式。這包括擷取指派給服務的外部 IP 位址，並透過簡單的前端與其互動。

擷取服務的外部 IP

取得公開服務的外部 IP (佈建作業約需 30 秒)

```
export IP_ADDRESS=$(kubectl get service "${GKE_DEPLOY}" -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo $IP_ADDRESS
```

在本機執行前端

在本示範中，前端是簡單的 HTML/JavaScript 網頁，不屬於 GKE 部署作業。這項工具專為在本機電腦上執行而設計。

在正式版應用程式中，前端應從 GCP 提供服務。在本機電腦上：

```
# Update the index.html file with the server IP address
cd frontend

# For Linux
sed -i "s#\(\const BACKEND_URL = \"http://\)[^/]+\(\(/analyze\";\)\)#\1${IP_ADDRESS}\2#g"
"index.html"

# For MacOS
#sed -i '' "s#\(\const BACKEND_URL = #\"http://\)[^/]*\(\(/analyze\";\)\)#\1${IP_ADDRESS}\2#g"
"index.html"

python3 -m http.server 8001
```

在 Chrome 中開啟 <http://localhost:8001/index.html>。

9. 清除

如要避免產生持續性費用，請刪除本程式碼研究室中建立的資源。

毀損 GKE 叢集

如要刪除整個應用程式，只要刪除 GKE 叢集即可。如要這麼做，請執行下列指令：

```
gcloud container clusters delete "${GKE_CLUSTER}" \
  --region="${REGION}" \
  --project="${PROJECT_ID}"
```

刪除 Assured Workloads

執行下列指令，刪除所有與 Assured Workload 相關的資源。

```
# Workload project deletion
gcloud billing projects unlink "${PROJECT_ID}"
gcloud projects delete "${PROJECT_ID}"

# KMS project deletion
gcloud billing projects unlink "${KMS_PROJECT_ID}"
gcloud projects delete "${KMS_PROJECT_ID}"

# Assured Workload folder deletion
gcloud resource-manager folders delete ${WORKLOAD_FOLDER_ID} --quiet

# Assured Workload deletion
gcloud assured workloads delete "${ASSURED_WORKLOAD_ID}" \
  --location="${REGION}" \
  --organization="${GCP_ORGANIZATION}" \
  --project="${QUOTA_PROJECT_ID}"
```

步驟 10 · 約 5 分鐘

10. 恭喜

任務完成！ 您已成功為受監管產業建構工作場所安全偵測系統，使用 Gemini 偵測安全帽。

完成的目標：

- **資料保護和隱私權：** 您已使用 CMEK 佈建機密 GKE 節點
- **資料邊界：** 您已為受監管的环境啟用平台法規遵循控制選項

參考文件

- [以機密 GKE 節點加密使用中的工作負載資料](#)
 - [使用自訂組織政策](#)
 - [透過 Assured Workloads 建立資料邊界總覽](#)
-